
Reading and Writing Spreadsheets and CSV Files in Python



Spreadsheets

- A **spreadsheet** is a digital document used to **organize data in rows and columns**.
- It allows users to **store, manipulate, and perform calculations on data** easily.
- Spreadsheets are **powerful tools for data analysis and management**.

Popular Spreadsheet Applications

- **Microsoft Excel**
- **Google Sheets**
- **Apple Numbers**
- **LibreOffice Calc**

CSV (Comma-Separated Values)

- CSV is a simple **text file format** used to store **tabular data**.
- It is widely used for data exchange because it is simple and easy to process.
- Data is organized in **rows and columns**, similar to a spreadsheet.
- Each **line in the file represents one row** of the table.
- Values in each row are separated by a **delimiter (separator)**.
- **Common Delimiters:** Comma (,) Semicolon (;)

Why CSV is Popular?

- Can be easily read and processed by programming languages **like Python**

Contd...

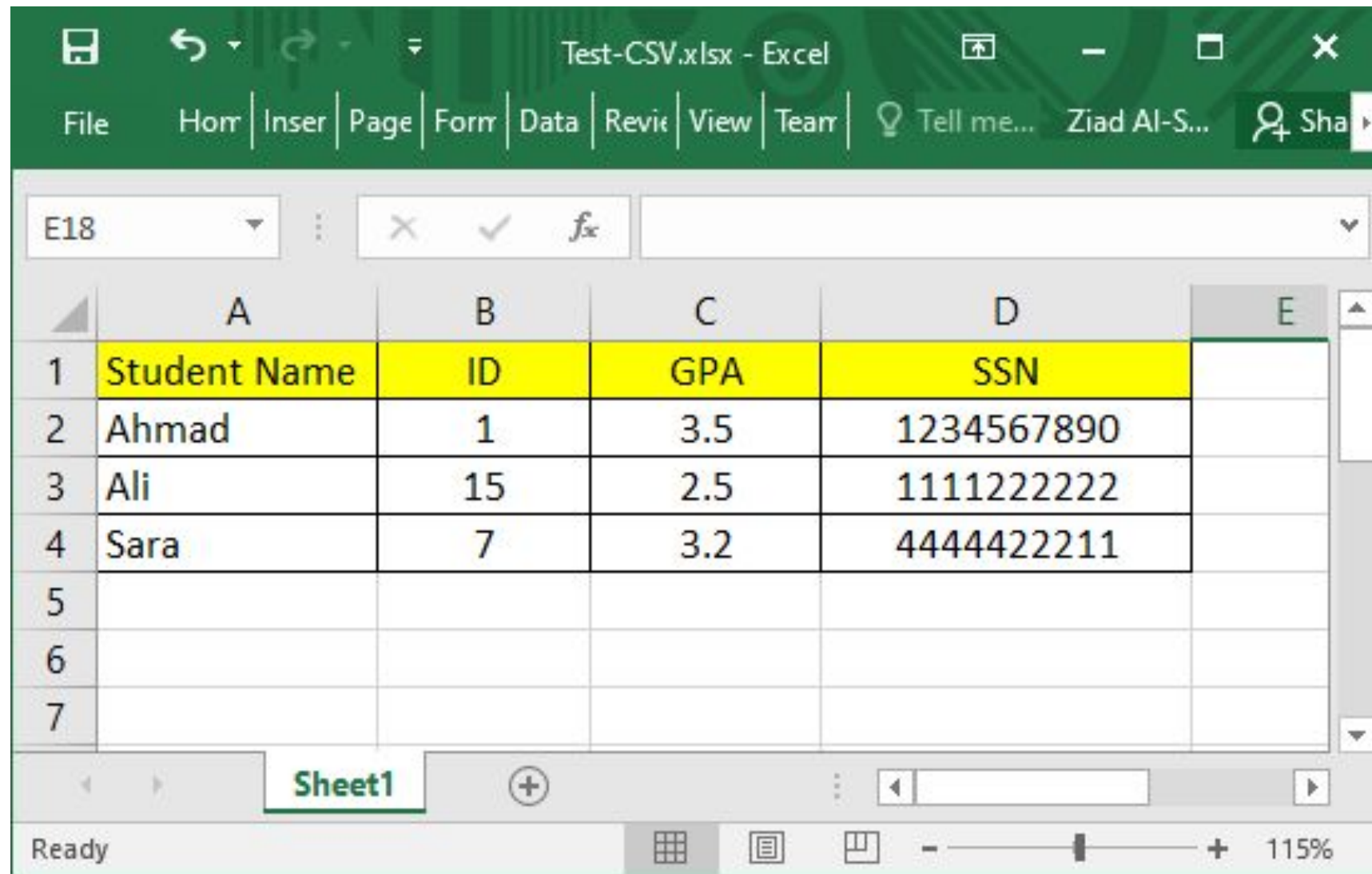
Structure of a CSV File

- Data is organized in rows and columns.
- Each row represents one record in the table.
- Each row ends with a newline to start the next row.
- Columns are separated by commas (,).

Why CSV is Widely Used

- Easy to create, read, and share.
- Many websites allow users to **export data as CSV files**.
- CSV files can be **opened in spreadsheet software like Microsoft Excel**.
- Most **database systems support importing data from CSV files**.

Spreadsheet and corresponding CSV file



	A	B	C	D	E
1	Student Name	ID	GPA	SSN	
2	Ahmad	1	3.5	1234567890	
3	Ali	15	2.5	1111222222	
4	Sara	7	3.2	4444422211	
5					
6					
7					

Student Name, ID, GPA, SSN

Ahmad, 1, 3.5, 1234567890

Ali, 15, 2.5, 1111222222

Sara, 7, 3.2, 4444422211

CSV, basic sharing

- A basic approach to share data is the comma separated value (CSV) format
- It is a text format, accessible to all apps
- each line (even if blank) is a row
- Values in a row are separated by commas (even if it is blank)
- cannot capture complex things like formula (unlike MS Excel)

Example

Table Data

Student Name	ID	GPA	SSN
Ahmad	1	3.5	1234567890
Ali	15	2.5	1111222222
Sara	7	3.2	4444422211

It can be
opened with
MS Excel

M18					
	A	B	C	D	E
1	Student Name	ID	GPA	SSN	
2	Ahmad	1	3.5	1234567890	
3	Ali	15	2.5	1111222222	
4	Sara	7	3.2	4444422211	
5					
6					
7					
8					

CSV Data

Student Name, ID, GPA, SSN
Ahmad, 1, 3.5, 1234567890
Ali, 15, 2.5, 1111222222
Sara, 7, 3.2, 4444422211

It can be opened with
Notepad or any simple
text editors

CSV Module in Python

- Although CSV is simple, different applications may have **small variations in format**.
- Python provides a built-in module called `csv` to handle these variations.

Steps to Read a CSV File in Python

1. Import the CSV module - `import csv`

2. Open the CSV file

Use the normal `open()` function to create a file object.

3. Create a CSV reader

The **reader object** reads data from the file.

4. Iterate through the file

Each row is returned as a list of strings.

Contd...

Opening a CSV File for Reading

```
fin = open('sample-in.csv', 'r')
```

- Opens the CSV file in read mode.
- Used to read data from an existing CSV file.

Opening a CSV File for Writing

```
fout = open('sample-out.csv', 'w', newline='')
```

- Opens a new CSV file in write mode.
- Used to write data into a CSV file.
- `newline= ''` is used for switching to next line(row) while entering data in row.

CSV Module

- `import csv` : import necessary modules
- `csv.reader()` : read data from a csv file
- `csv.writer()` : write data to a csv file

data.csv

cv-reader.py - C:/Users/zasha/OneDrive/Desktop/Python-Csv/cv-reader.py (3.9.0)

File Edit Format Run Options Window Help

```
import csv
with open('data.csv', 'rt') as f:
    data = csv.reader(f)
    for row in data:
        print(row)
```

Student Name,	ID,	GPA,	SSN
Ahmad,	1,	3.5,	1234567890
Ali,	15,	2.5,	1111222222
Sara,	7,	3.2,	4444422211

```
['Student Name', '\tID', '\tGPA', '\tSSN']
['Ahmad', '\t\t1', '\t3.5', '\t1234567890']
['Ali', '\t\t15', '\t2.5', '\t1111222222']
['Sara', '\t\t7', '\t3.2', '\t4444422211']
```

without csv.reader

```
f = open('data.csv', 'tr')
for line in f:
    print(line, end='')
f.close()
```

```
Student Name, ID, GPA, SSN
Ahmad, 1, 3.5, 1234567890
Ali, 15, 2.5, 1111222222
Sara, 7, 3.2, 4444422211
>>> |
```

Simply reads a simple line of text

with csv.reader

```
import csv
f = open('data.csv', 'tr')
csv_f = csv.reader(f)
for line in csv_f:
    print(line, end='\n')
f.close()
```

```
['Student Name', '\tID', '\tGPA', '\tSSN']
['Ahmad', '\t\t1', '\t3.5', '\t1234567890']
['Ali', '\t\t15', '\t2.5', '\t1111222222']
['Sara', '\t\t7', '\t3.2', '\t4444422211']
>>>
```

Read each row and form of list

csv.reader()

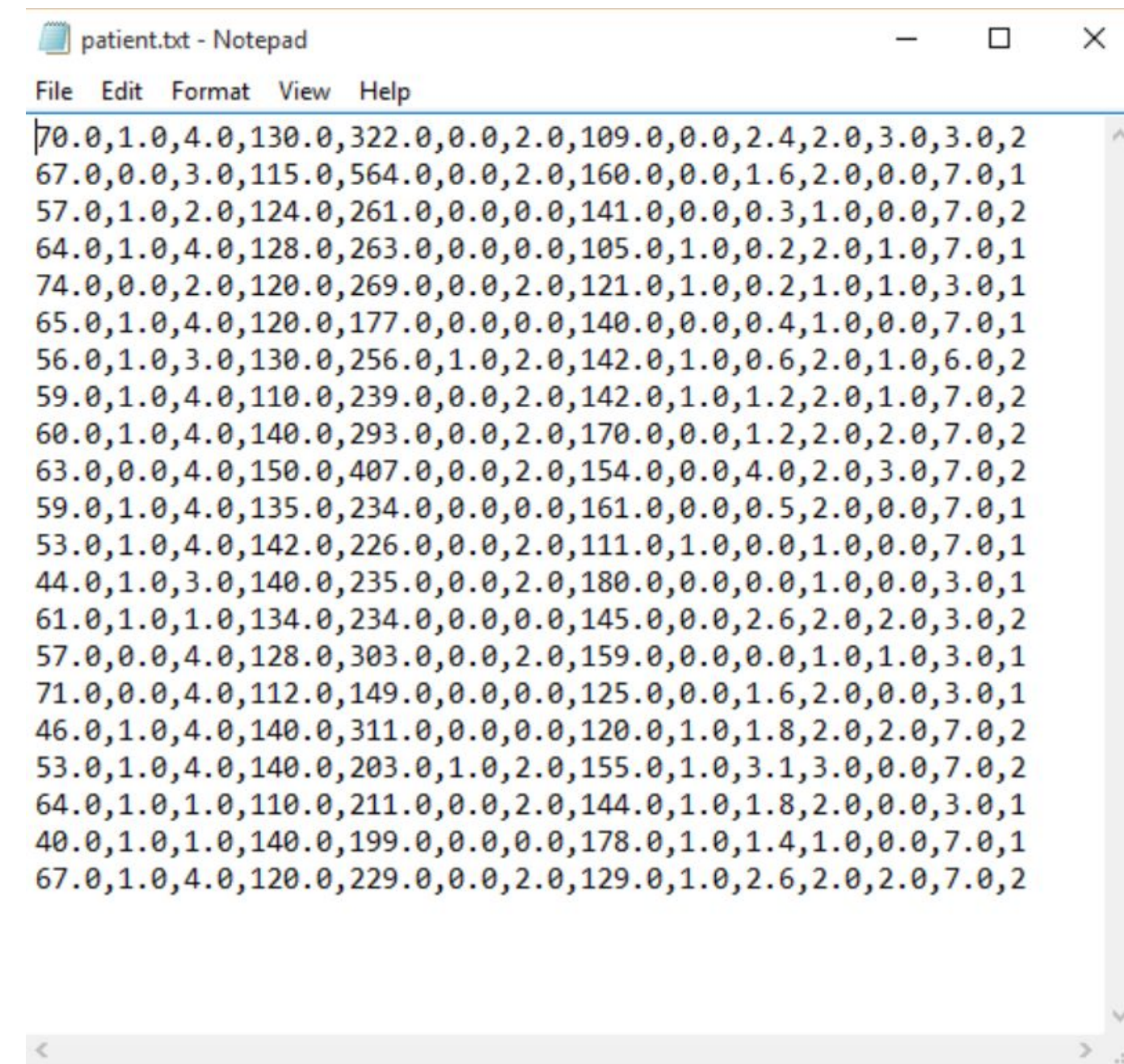
- First, we have to actually `import` the csv module
- We also have to open the file for reading
- Then we create a very simple object using the `csv.reader(f)` that accepts a file object.
- Inside the reader function, we pass the file object into the `csv.reader(f)` function, which returns a reader object.

Example #2

- **csv.reader** returns a reader object which will iterate over lines in the given csv file.

```
import csv
fr=open('patient.txt','r');
csv_f = csv.reader(fr)
for line in csv_f:
    print(line)
```

```
>>> ===== RESTART =====
>>>
['70.0', '1.0', '4.0', '130.0', '322.0', '0.0', '2.0', '109.0', '0.0', '2.4', '2.0', '3.0', '3.0', '2']
['67.0', '0.0', '3.0', '115.0', '564.0', '0.0', '2.0', '160.0', '0.0', '1.6', '2.0', '0.0', '7.0', '1']
['57.0', '1.0', '2.0', '124.0', '261.0', '0.0', '0.0', '141.0', '0.0', '0.3', '1.0', '0.0', '7.0', '2']
['64.0', '1.0', '4.0', '128.0', '263.0', '0.0', '0.0', '105.0', '1.0', '0.2', '2.0', '1.0', '7.0', '1']
['74.0', '0.0', '2.0', '120.0', '269.0', '0.0', '2.0', '121.0', '1.0', '0.2', '1.0', '1.0', '3.0', '1']
['65.0', '1.0', '4.0', '120.0', '177.0', '0.0', '0.0', '140.0', '0.0', '0.4', '1.0', '0.0', '7.0', '1']
['56.0', '1.0', '3.0', '130.0', '256.0', '1.0', '2.0', '142.0', '1.0', '0.6', '2.0', '1.0', '6.0', '2']
['59.0', '1.0', '4.0', '110.0', '239.0', '0.0', '2.0', '142.0', '1.0', '1.2', '2.0', '1.0', '7.0', '2']
['60.0', '1.0', '4.0', '140.0', '293.0', '0.0', '2.0', '170.0', '0.0', '1.2', '2.0', '2.0', '7.0', '2']
['63.0', '0.0', '4.0', '150.0', '407.0', '0.0', '2.0', '154.0', '0.0', '4.0', '2.0', '3.0', '7.0', '2']
['59.0', '1.0', '4.0', '135.0', '234.0', '0.0', '0.0', '161.0', '0.0', '0.5', '2.0', '0.0', '7.0', '1']
['53.0', '1.0', '4.0', '142.0', '226.0', '0.0', '2.0', '111.0', '1.0', '0.0', '1.0', '0.0', '7.0', '1']
['44.0', '1.0', '3.0', '140.0', '235.0', '0.0', '2.0', '180.0', '0.0', '0.0', '1.0', '0.0', '3.0', '1']
['61.0', '1.0', '1.0', '134.0', '234.0', '0.0', '0.0', '145.0', '0.0', '2.6', '2.0', '2.0', '3.0', '2']
['57.0', '0.0', '4.0', '128.0', '303.0', '0.0', '2.0', '159.0', '0.0', '0.0', '1.0', '1.0', '3.0', '1']
['71.0', '0.0', '4.0', '112.0', '149.0', '0.0', '0.0', '125.0', '0.0', '1.6', '2.0', '0.0', '3.0', '1']
['46.0', '1.0', '4.0', '140.0', '311.0', '0.0', '0.0', '120.0', '1.0', '1.8', '2.0', '2.0', '7.0', '2']
['53.0', '1.0', '4.0', '140.0', '203.0', '1.0', '2.0', '155.0', '1.0', '3.1', '3.0', '0.0', '7.0', '2']
['64.0', '1.0', '1.0', '110.0', '211.0', '0.0', '2.0', '144.0', '1.0', '1.8', '2.0', '0.0', '3.0', '1']
['40.0', '1.0', '1.0', '140.0', '199.0', '0.0', '0.0', '178.0', '1.0', '1.4', '1.0', '0.0', '7.0', '1']
['67.0', '1.0', '4.0', '120.0', '229.0', '0.0', '2.0', '129.0', '1.0', '2.6', '2.0', '2.0', '7.0', '2']
```



```
patient.txt - Notepad
File Edit Format View Help
70.0,1.0,4.0,130.0,322.0,0.0,2.0,109.0,0.0,2.4,2.0,3.0,3.0,2
67.0,0.0,3.0,115.0,564.0,0.0,2.0,160.0,0.0,1.6,2.0,0.0,7.0,1
57.0,1.0,2.0,124.0,261.0,0.0,0.0,141.0,0.0,0.3,1.0,0.0,7.0,2
64.0,1.0,4.0,128.0,263.0,0.0,0.0,105.0,1.0,0.2,2.0,1.0,7.0,1
74.0,0.0,2.0,120.0,269.0,0.0,2.0,121.0,1.0,0.2,1.0,1.0,3.0,1
65.0,1.0,4.0,120.0,177.0,0.0,0.0,140.0,0.0,0.4,1.0,0.0,7.0,1
56.0,1.0,3.0,130.0,256.0,1.0,2.0,142.0,1.0,0.6,2.0,1.0,6.0,2
59.0,1.0,4.0,110.0,239.0,0.0,2.0,142.0,1.0,1.2,2.0,1.0,7.0,2
60.0,1.0,4.0,140.0,293.0,0.0,2.0,170.0,0.0,1.2,2.0,2.0,7.0,2
63.0,0.0,4.0,150.0,407.0,0.0,2.0,154.0,0.0,4.0,2.0,3.0,7.0,2
59.0,1.0,4.0,135.0,234.0,0.0,0.0,161.0,0.0,0.5,2.0,0.0,7.0,1
53.0,1.0,4.0,142.0,226.0,0.0,2.0,111.0,1.0,0.0,1.0,0.0,7.0,1
44.0,1.0,3.0,140.0,235.0,0.0,2.0,180.0,0.0,0.0,1.0,0.0,3.0,1
61.0,1.0,1.0,134.0,234.0,0.0,0.0,145.0,0.0,2.6,2.0,2.0,3.0,2
57.0,0.0,4.0,128.0,303.0,0.0,2.0,159.0,0.0,0.0,1.0,1.0,3.0,1
71.0,0.0,4.0,112.0,149.0,0.0,0.0,125.0,0.0,1.6,2.0,0.0,3.0,1
46.0,1.0,4.0,140.0,311.0,0.0,0.0,120.0,1.0,1.8,2.0,2.0,7.0,2
53.0,1.0,4.0,140.0,203.0,1.0,2.0,155.0,1.0,3.1,3.0,0.0,7.0,2
64.0,1.0,1.0,110.0,211.0,0.0,2.0,144.0,1.0,1.8,2.0,0.0,3.0,1
40.0,1.0,1.0,140.0,199.0,0.0,0.0,178.0,1.0,1.4,1.0,0.0,7.0,1
67.0,1.0,4.0,120.0,229.0,0.0,2.0,129.0,1.0,2.6,2.0,2.0,7.0,2
```

How to Read a CSV as a Dictionary

- You can also use DictReader to read CSV files.

csv-DictReader.py - C:/Users/zasha/OneDrive/Desktop/Python-CSV/csv-DictReader.py (3.9.0)

File Edit Format Run Options Window Help

```
#import necessary modules
import csv
```

```
reader = csv.DictReader(open("data.csv"))
for row in reader:
    print(row)
```

```
= RESTART: C:/Users/zasha/OneDrive/Desktop/Python-CSV/csv-DictReader.py
{'Student Name': 'Ahmad', '\tID': '\t\t1', '\tGPA': '\t3.5', '\tSSN': '\t1234567890'}
{'Student Name': 'Ali', '\tID': '\t\t15', '\tGPA': '\t2.5', '\tSSN': '\t1111222222'}
{'Student Name': 'Sara', '\tID': '\t\t7', '\tGPA': '\t3.2', '\tSSN': '\t4444422211'}
>>>
```

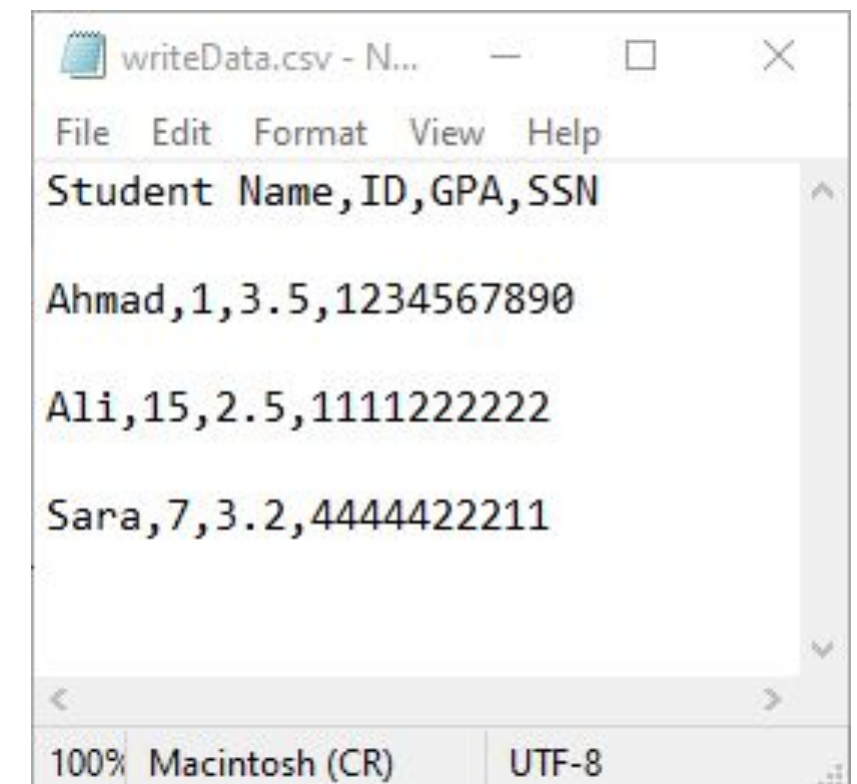
How to write CSV File

- When you have a set of data that you would like to store in a CSV file, you have to use **writer()** function.
- To iterate the data over the rows(lines), you have to use the **writerow()** function.

```
#import necessary modules
import csv

with open('writeData.csv', 'tw') as file:
    writer = csv.writer(file, delimiter=',', quotechar='"')

    #way to write to csv file
    writer.writerow(['Student Name', 'ID', 'GPA', 'SSN'])
    writer.writerow(['Ahmad', '1', '3.5', '1234567890'])
    writer.writerow(['Ali', '15', '2.5', '1111222222'])
    writer.writerow(['Sara', '7', '3.2', '4444422211'])
```



CSV Module vs Pandas

Feature	csv module	pandas
Reading CSV	Yes	Yes
Data analysis	No	Yes
Statistics	No	Yes
Filtering	Difficult	Easy
Large datasets	Hard	Efficient

Reading and Writing CSV Files with Pandas



Pandas

- **Pandas is a powerful Python library for data analysis and data manipulation.**
- It provides **fast, flexible, and easy-to-use data structures** for working with structured data.
- Helps in **analyzing, cleaning, and organizing data** efficiently.
- Widely used for **real-world data analysis in Python.**

Types of Data Pandas Can Handle

- **Tabular data** (like an SQL table or spreadsheet in Microsoft Excel)
- **Time series data** (ordered or unordered data over time)
- **Matrix data** with row and column labels
- **Statistical or observational datasets**

Contd...

- The word Pandas comes from **Python + Data Analysis** (also related to **Panel Data**).
- It is built on top of **NumPy**, which provides **fast numerical computations**.
- Other libraries like **SciPy** and **Matplotlib** are optional but very useful with Pandas.

Main Data Structures in Pandas

- Series : 1-Dimensional data
- DataFrame : 2-Dimensional table-like data
- Widely used in **data analysis, statistics, finance, and engineering applications**.

Series

- A **Series** is a **one-dimensional labeled array-like data structure** in Pandas.
- A Series represents a **single column of data with labels**.
- It can store **different types of data**, such as:
 - **Integers**
 - **Floats**
 - **Strings**
 - **Python objects**

Structure of a Series

- One **array for the index** (labels)
- One **array containing the actual data values**

Contd...

Data structures

Dimensions	Name	Description
1	Series	1D labeled homogeneously-typed array
2	DataFrame	General 2D labeled, size-mutable tabular structure with potentially heterogeneously-typed column

Contd...

- The two main components of Pandas are **Series** and **DataFrame**.
- A Series is a **one-dimensional data structure**, similar to a **single column of data**.
- A DataFrame is a **two-dimensional table**, made up of **multiple Series combined together**.
 - **Series** → One column of data
 - **DataFrame** → A table containing many columns

Series

	apples
0	3
1	2
2	0
3	1

+

Series

	oranges
0	0
1	3
2	7
3	2

=

DataFrame

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

Series

- Series is like a one-dimensional array like structure with homogeneous data. For example, the following series is a collection of integers.

Basic feature of series are

- ❖ Homogeneous data
- ❖ Size Immutable
- ❖ Values of Data Mutable

78	45	12	89	56
----	----	----	----	----

Creating a Pandas Series

- A **Series** is a **one-dimensional array-like structure** that can store data of any type (integer, float, string, Python objects, etc.).
- A Series is created using the **constructor**:

Syntax: `pandas.Series(data, index, dtype, copy)`

- A Series can be **created from different sources**, such as:
 - **Array**
 - **Dictionary**
 - **Scalar value (constant)**
- It provides **labeled data with an index for each value**.

Contd...

Example:

```
import pandas as pd  
  
s = pd.Series()  
print(s)
```

Output:

```
Series([], dtype: float64)
```

- This creates an empty Series with no elements.
- The default data type (dtype) is float64.

Creating a Series from an Array

- A Pandas Series can be created from a NumPy array.

Example:

```
import pandas as pd
import numpy as np

data = np.array(['a', 'b', 'c', 'd'])
s = pd.Series(data)
print(s)
```

Output:

```
0    a
1    b
2    c
3    d
dtype: object
```

Note:

- The default index starts from 0.
- We can access:
 - Index values → `print(s.index)`
 - Data values → `print(s.values)`

Creating a Series from a Dictionary

- A Pandas Series can also be created using a dictionary.

Example:

```
import pandas as pd
import numpy as np

data = np.array(['a', 'b', 'c', 'd'])
s = pd.Series(data)
print(s)
```

Output:

```
0    a
1    b
2    c
3    d
dtype: object
```

Note:

- The default index starts from 0.
- We can access:
 - Index values → `print(s.index)`
 - Data values → `print(s.values)`

Contd...

Example:

```
import pandas as pd

data = {'a': 0., 'b': 1., 'c': 2.}
s = pd.Series(data)

print(s)
```

Output:

```
a    0.0
b    1.0
c    2.0
dtype: float64
```

Note:

- The keys of the dictionary become the index (labels).
- The values of the dictionary become the data in the Series.

Mathematical Operations on Pandas Series

- Pandas allows element-wise mathematical operations on Series objects.
- If two Series have the **same index**, operations are performed **element by element**.

Example: Addition and Multiplication

```
import pandas as pd

# Creating two Series
s = pd.Series([1, 2, 3])
t = pd.Series([1, 2, 4])

# Addition of two Series
u = s + t
print(u)

# Multiplication of two Series
u = s * t
print(u)
```

Output: Addition (s + t)

```
0 2
1 4
2 7
dtype: int64
```

Multiplication (s * t)

```
0 1
1 4
2 12
dtype: int64
```

Pandas Series

- To create a Series with a custom index and view the index labels:

```
cities = ['Kolkata', 'Chicago', 'Toronto', 'Lisbon']  
populations = [14.85, 2.71, 2.93, 0.51]  
city_series = pd.Series(populations, index=cities)  
print(city_series.index)
```

Output:

```
Index(['Kolkata', 'Chicago', 'Toronto', 'Lisbon'],  
      dtype='object')
```

Exercise - Pandas Series

- Create a Series with values: [10, 20, 30, 40, 50]

Compute:

- Find the mean (average)
- Find the standard deviation
- Multiply all values by 2

Pandas Series

- Create a Series with values: [10, 20, 30, 40, 50]

Compute:

- Find the mean (average)
- Find the standard deviation
- Multiply all values by 2

```
import pandas as pd
data = pd.Series([10, 20, 30, 40, 50])
print("Mean:", data.mean())
print("Standard Deviation:", data.std())
data = data * 2
print("Updated Series:")
print(data)
```

DataFrame

- DataFrame is like a two-dimensional array or table that can store heterogeneous data (different data types) arranged in rows and columns.

SR. No.	Admn No	Student Name	Class	Section	Gender	Date Of Birth
1	001284	NIDHI MANDAL	I	A	Girl	07/08/2010
2	001285	SOUMYADIP BHATTACHARYA	I	A	Boy	24/02/2011
3	001286	SHREYAANG SHANDILYA	I	A	Boy	29/12/2010

Contd...

- **Types of Data in a DataFrame**

Numbers

Strings (Text)

Dates

Boolean values

Basic Features

- **Heterogeneous Data**

A DataFrame can store **different data types in different columns**.

- **Size Mutable**

The **size of a DataFrame can change**. Rows and columns can be **added or removed**.

- **Data Mutable**

The **values inside a DataFrame can be modified** after it is created.

Creating a DataFrame

- A **DataFrame** is a two-dimensional labeled data structure in Pandas.
- A DataFrame can be created from the following data sources:
 - Lists
 - Dictionary (dict)
 - Pandas Series
 - NumPy ndarrays
 - Another DataFrame

Creating an Empty DataFrame

Example:

```
import pandas as pd

df1 = pd.DataFrame()

print(df1)
```

Output:

```
Empty DataFrame
Columns: []
Index: []
```

Note:

- A DataFrame created without data contains no rows and no columns.

Create a DataFrame from a List

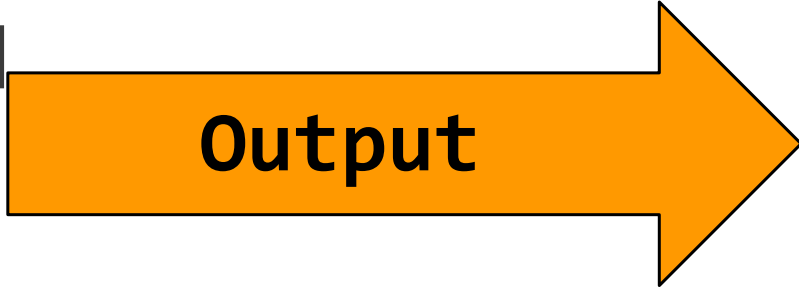
Example:

```
import pandas as pd

data1 = [10, 20, 30, 40, 50]

df1 = pd.DataFrame(data1)

print(df1)
```



Output

	0
0	10
1	20
2	30
3	40
4	50

Note:

- A simple list becomes a single column DataFrame
- Pandas automatically creates the index

Create a DataFrame from a List of Lists

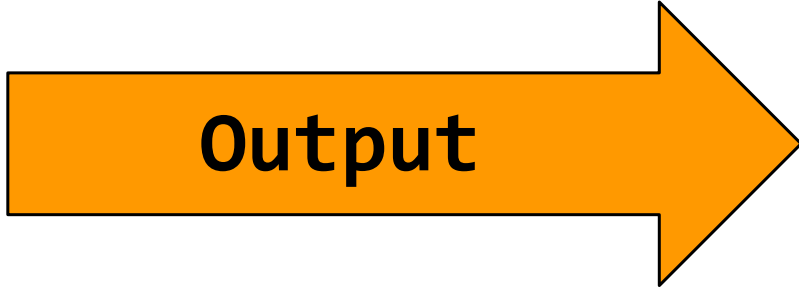
Example:

```
import pandas as pd
```

```
data1 = [['Freya', 10],  
         ['Mohak', 12],  
         ['Dwivedi', 13]]
```

```
df1 = pd.DataFrame(data1, columns=['Name', 'Age'])
```

```
print(df1)
```



Output

	Name	Age
0	Freya	10
1	Mohak	12
2	Dwivedi	13

Note:

- Each inner list represents a row
- Column names can be specified using columns

Outer list → rows

Inner list → values in each row

Create a DataFrame from a Dictionary

Example:

```
import pandas as pd

data = {
    'apples': [3, 2, 0, 1],
    'oranges': [0, 3, 7, 2]
}

purchases = pd.DataFrame(data)
print(purchases)
```

Output

	apples	oranges
0	3	0
1	2	3
2	0	7
3	1	2

Note:

- Dictionary keys → Column names
- List values → Column data
- Pandas automatically creates the index

Dictionary Key → DataFrame Column

Dictionary Value → Column Data

'apples' → column

[3,2,0,1] → values in that column

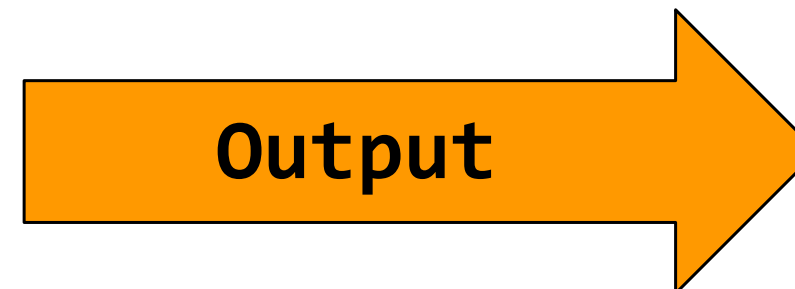
Creating a Custom Index

- By default, Pandas assigns numeric indices **0,1,2,3...**
- We can also specify **our own index labels**.

Example:

```
import pandas as pd

data = {
    'apples': [3, 2, 0, 1],
    'oranges': [0, 3, 7, 2]
}
```



	apples	oranges
June	3	0
Robert	2	3
Lily	0	7
David	1	2

```
purchases = pd.DataFrame(data, index=['June', 'Robert', 'Lily', 'David'])
```

```
print(purchases)
```

Columns → describe features

Index → identifies rows

Selecting Data in Pandas: loc vs iloc

- So now we could locate a customer's order by using their name:

```
purchases.loc['June']
```

```
purchases.iloc[0]
```

- Pandas provides two methods to select data from a DataFrame:

Method	Meaning
loc	Select using labels (names)
iloc	Select using index positions

Exercise - Pandas DataFrame

- Create a DataFrame with:
- Name: ['A', 'B', 'C']
- Marks: [60, 75, 90]
- Compute:
- Add a new column $\text{Percentage} = \text{Marks} / 100 * 100$

Contd...

- Create a DataFrame with:
- Name: ['A', 'B', 'C']
- Marks: [60, 75, 90]
- Compute:
- Add a new column $\text{Percentage} = \text{Marks} / 100 * 100$
- Find students who scored above average
- Print only their names

Contd...

```
import pandas as pd
data = {
    'Name': ['A', 'B', 'C'],
    'Marks': [60, 75, 90]
}
df = pd.DataFrame(data)
df['Percentage'] = df['Marks'] / 100 * 100
avg = df['Marks'].mean()
print("Students above average:")
print(df[df['Marks'] > avg]['Name'])
```

Reading a CSV File into a Pandas DataFrame

- `read_csv()` reads a CSV file into a DataFrame
- Pandas automatically detects columns and rows
- The result is stored in a DataFrame object

```
1 #import necessary modules
2 import pandas
3 result = pandas.read_csv('data.csv')
4 print(type(result))
5 print(result)
```

```
<class 'pandas.core.frame.DataFrame'>
  Student Name  \tID  \tGPA  \tSSN
0      Ahmad    1    3.5    1234567890
1       Ali    15    2.5    1111222222
2       Sara     7    3.2    4444422211
>>>
```

Writing to a CSV File with Pandas

- A Pandas DataFrame can be saved to a CSV file using `to_csv()`.

Example:

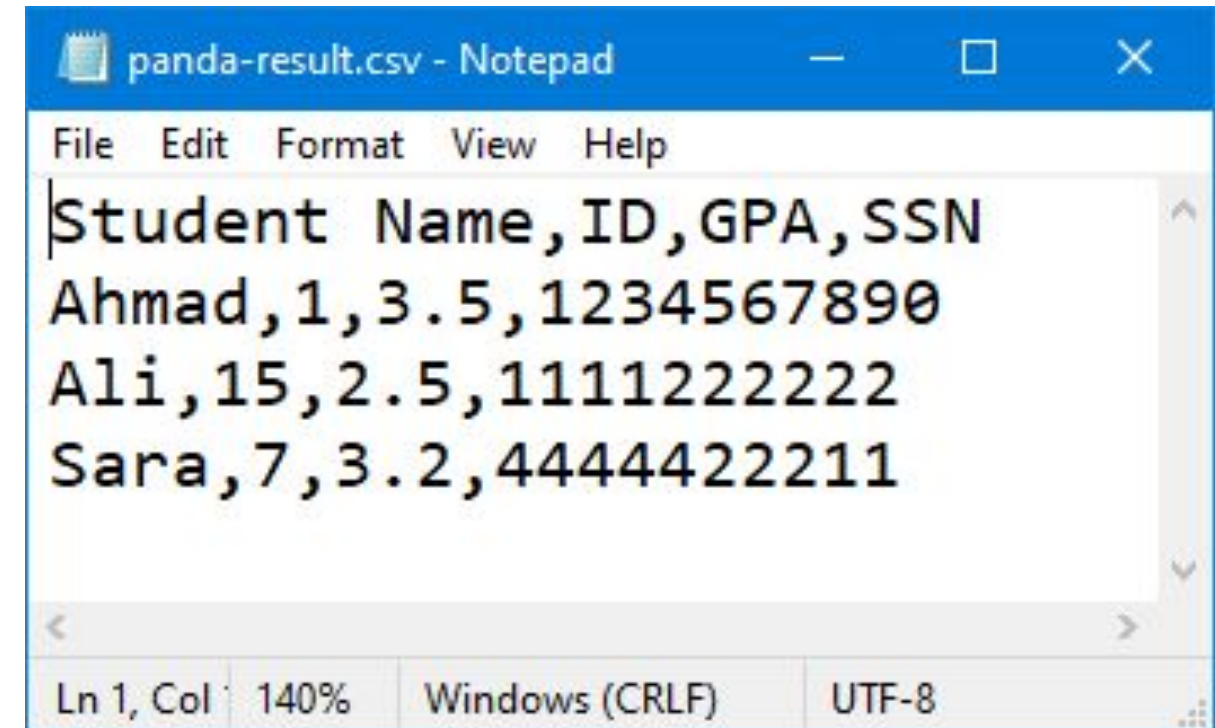
```
import pandas as pd

data = {
    'Student_Name': ['Ahmad', 'Ali', 'Sara'],
    'ID': [1, 15, 7],
    'GPA': [3.5, 2.5, 3.2],
    'SSN': [1234567890, 1111222222, 4444422211]
}

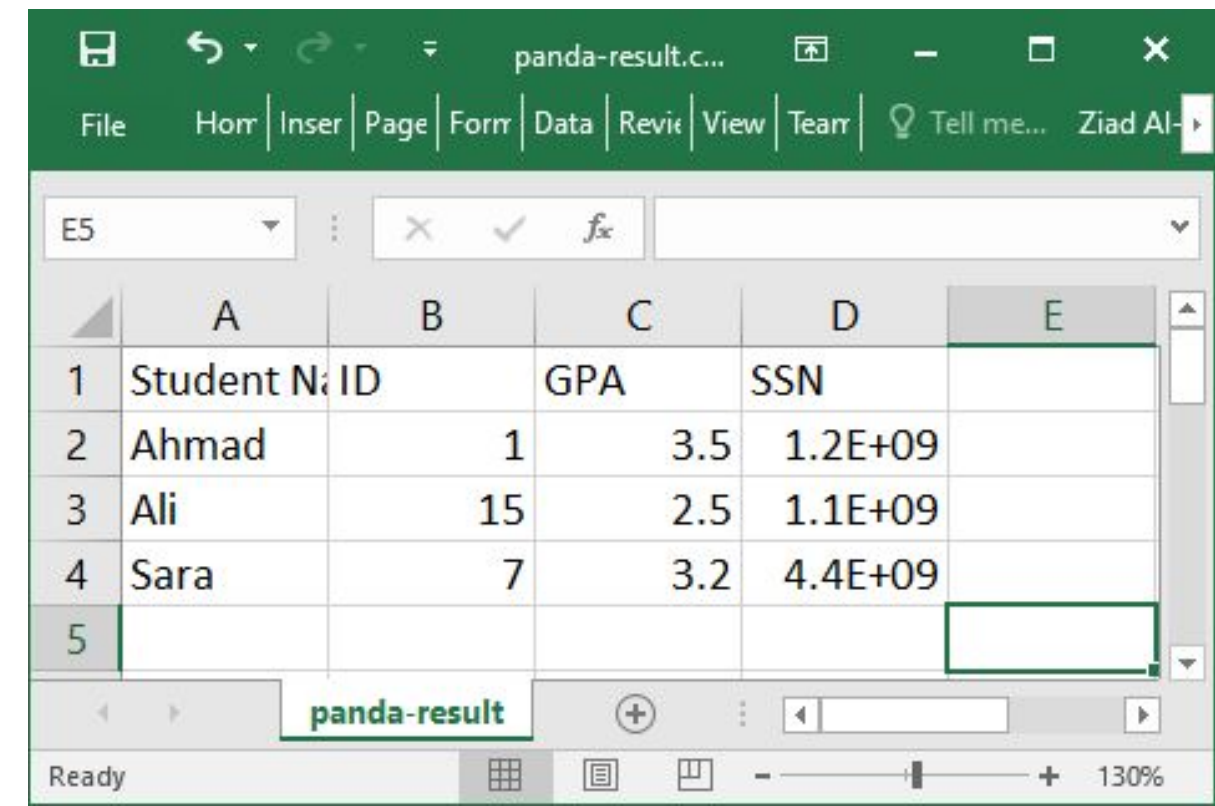
df = pd.DataFrame(data)

# Writing DataFrame to CSV
df.to_csv("panda-result.csv", index=False)

print("File saved successfully")
```



The screenshot shows a Notepad window titled 'panda-result.csv - Notepad'. The text inside is a CSV file with four columns: Student Name, ID, GPA, and SSN. The data rows are: Ahmad, 1, 3.5, 1234567890; Ali, 15, 2.5, 1111222222; and Sara, 7, 3.2, 4444422211. The status bar at the bottom indicates 'Ln 1, Col 140%', 'Windows (CRLF)', and 'UTF-8'.



The screenshot shows a spreadsheet application with a table containing the same data as the CSV file. The table has five columns labeled A, B, C, D, and E. The data rows are: 1, Student Name, ID, GPA, SSN; 2, Ahmad, 1, 3.5, 1.2E+09; 3, Ali, 15, 2.5, 1.1E+09; 4, Sara, 7, 3.2, 4.4E+09; and 5, which is empty. The status bar at the bottom indicates 'Ready' and '130%' zoom.

	A	B	C	D	E
1	Student Name	ID	GPA	SSN	
2	Ahmad	1	3.5	1.2E+09	
3	Ali	15	2.5	1.1E+09	
4	Sara	7	3.2	4.4E+09	
5					

Contd...

- `to_csv()` writes the DataFrame to a CSV file
- `index=False` prevents Pandas from writing row indices
- The CSV file can be opened in **Excel, Notepad, or any spreadsheet software**

Note:

	Function
Read CSV	<code>pd.read_csv()</code>
Write CSV	<code>df.to_csv()</code>

Read a .CSV File

Example:

```
import pandas as pd

# Read the CSV file
df = pd.read_csv("iris.csv")

# Display first 5 rows
print(df.head())

# Display dataset shape
print(df.shape)

# Display column names
print(df.columns)

# Display statistical summary
print(df.describe())
```

Function	Purpose
df.head()	Shows the first 5 rows of the dataset
df.shape	Shows (rows, columns)
df.columns	Shows column names
df.describe()	Shows statistical summary of numeric data

Example Output:

```
Shape
(150, 5)

Columns
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width', 'species'])
```

Data Visualization with **Matplotlib**



Data Visualization with Matplotlib

- Visualize the relationship between sepal length and petal length.

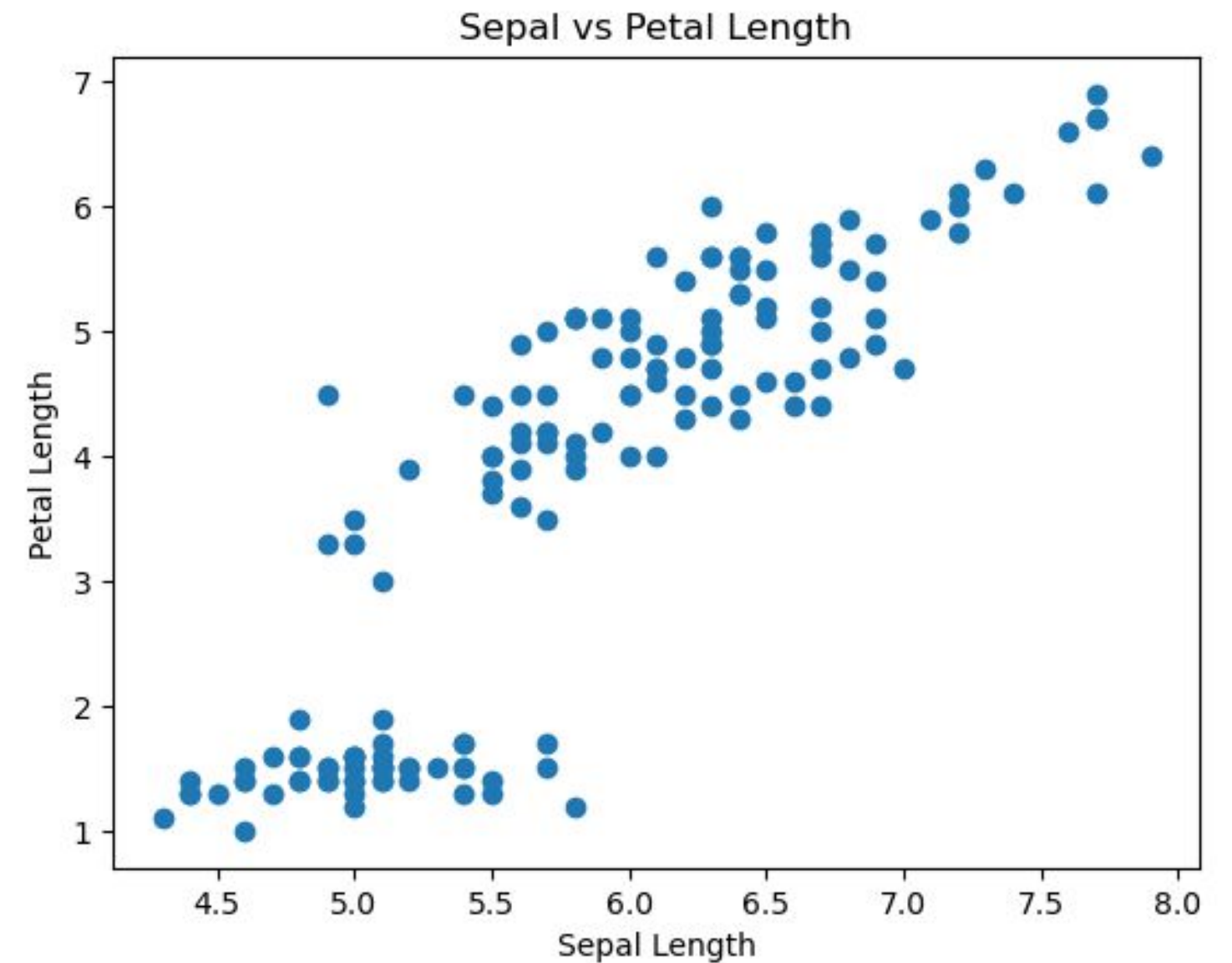
```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("/home/user/Downloads/iris.csv")

plt.scatter(df['sepal.length'], df['petal.length'])

plt.xlabel("Sepal Length")
plt.ylabel("Petal Length")
plt.title("Sepal vs Petal Length")

plt.show()
```



Histogram

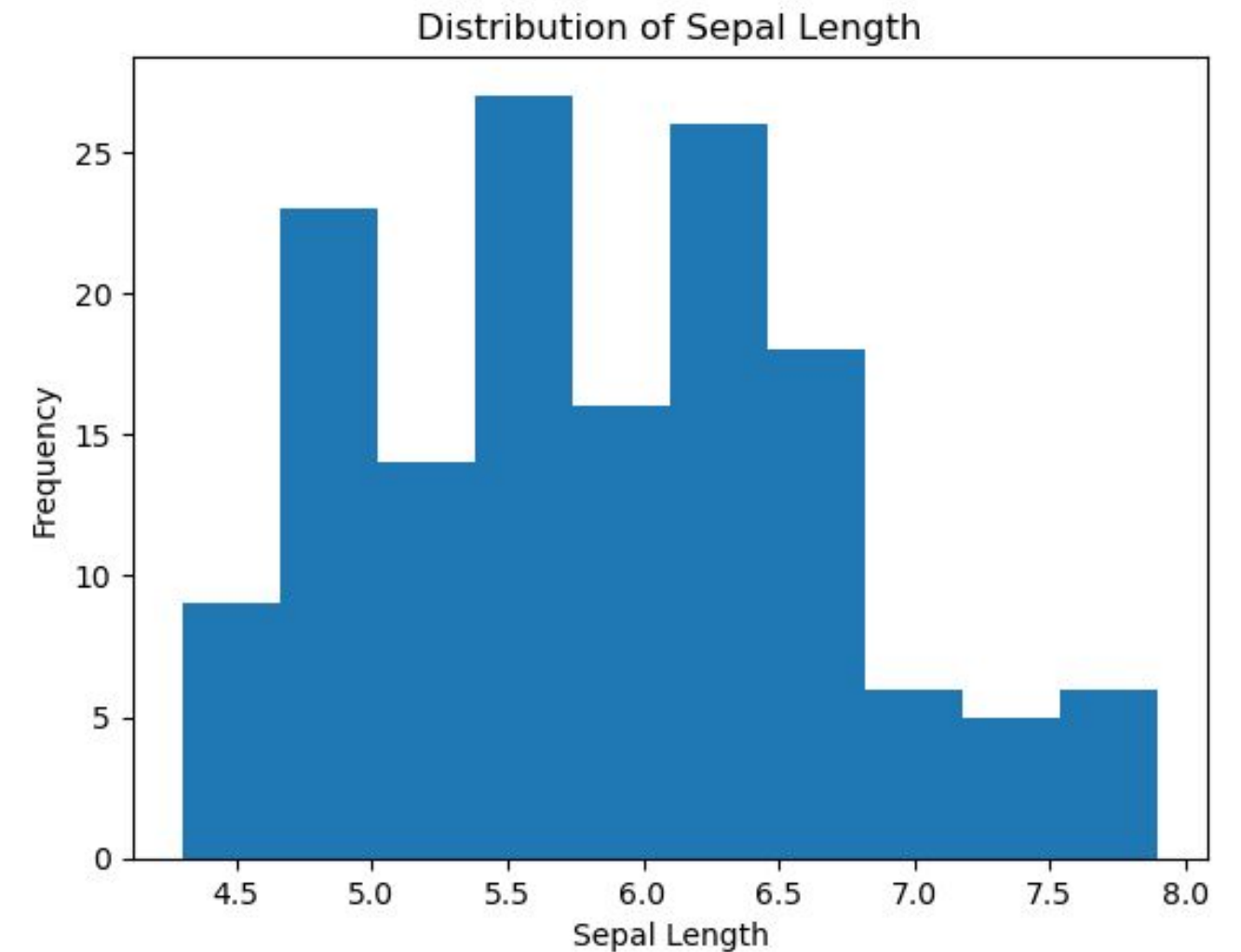
- Visualize the distribution of sepal length values.

```
import matplotlib.pyplot as plt

# Histogram
plt.hist(df['sepal.length'], bins=10)

# Labels and title
plt.xlabel("Sepal Length")
plt.ylabel("Frequency")
plt.title("Distribution of Sepal Length")

# Display plot
plt.show()
```



Scatter Plot

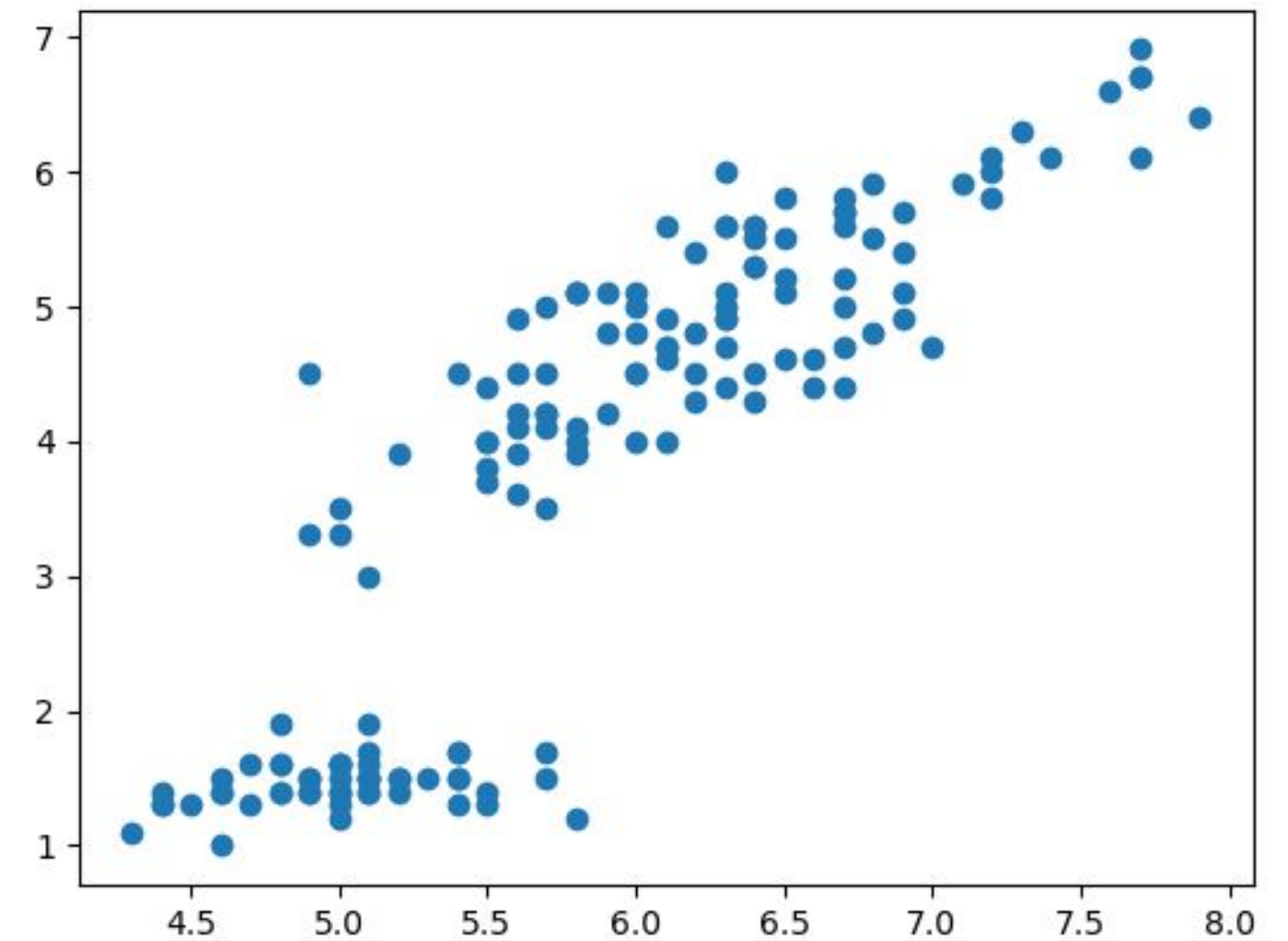
- Visualize the distribution of sepal length values.

```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
df = pd.read_csv("iris.csv")

# Scatter plot
plt.scatter(df['sepal.length'], df['petal.length'])

# Display plot
plt.show()
```



Using `df.plot()` for Scatter Plot

```
df.plot(kind="scatter", x="sepal.length", y="petal.length")  
plt.show()
```

- `df.plot(...)` → Uses **pandas built-in plotting** (internally uses Matplotlib)
- `kind="scatter"` → Specifies a scatter plot
- This is just a shortcut way to create the same scatter plot using pandas instead of writing full Matplotlib code.

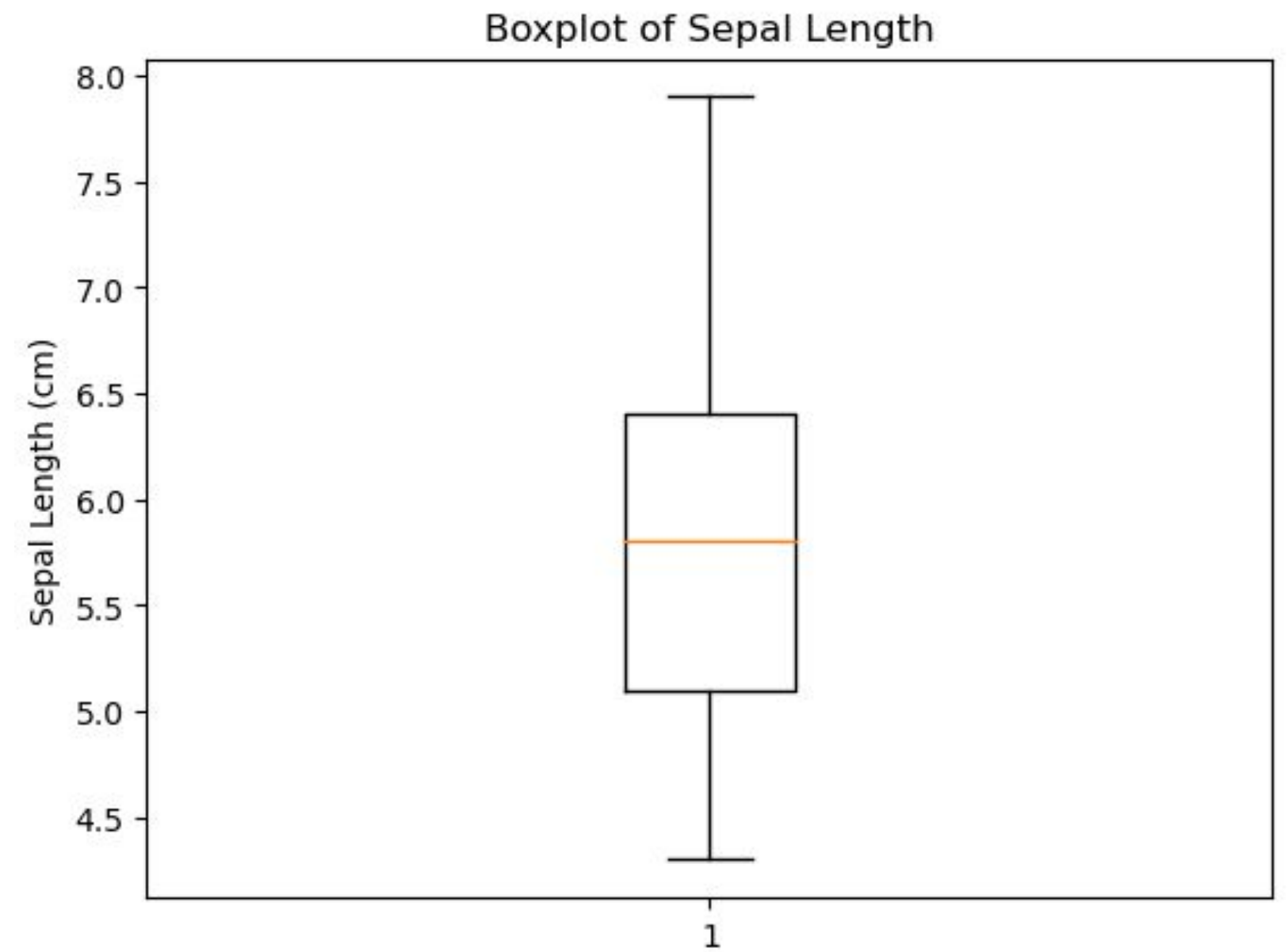
Box Plot

```
plt.boxplot(df["sepal.length"])
```

```
plt.ylabel("Sepal Length (cm)")
```

```
plt.title("Boxplot of Sepal Length")
```

```
plt.show()
```





```
import pandas as pd
```

```
df = pd.read_csv("iris.csv")
```

```
# Using loc (label based)
```

```
print(df.loc[0, "sepal.length"])
```

```
# Using iloc (position based)
```

```
print(df.iloc[0, 0])
```

```
df.loc[row_label, column_label]
```

```
df.iloc[row_position, column_position]
```

Handling Missing Values in Pandas

- Real-world datasets often contain missing or null values that can affect analysis.

Example: Detecting and handling missing data

```
import pandas as pd
```

```
df = pd.read_csv("iris.csv")
```

```
# Check missing values
```

```
print(df.isnull().sum())
```

```
# Remove rows with missing values
```

```
df_clean = df.dropna()
```

```
# Fill missing values only for numeric columns
```

```
df_filled = df.fillna(df.select_dtypes(include='number').mean())
```

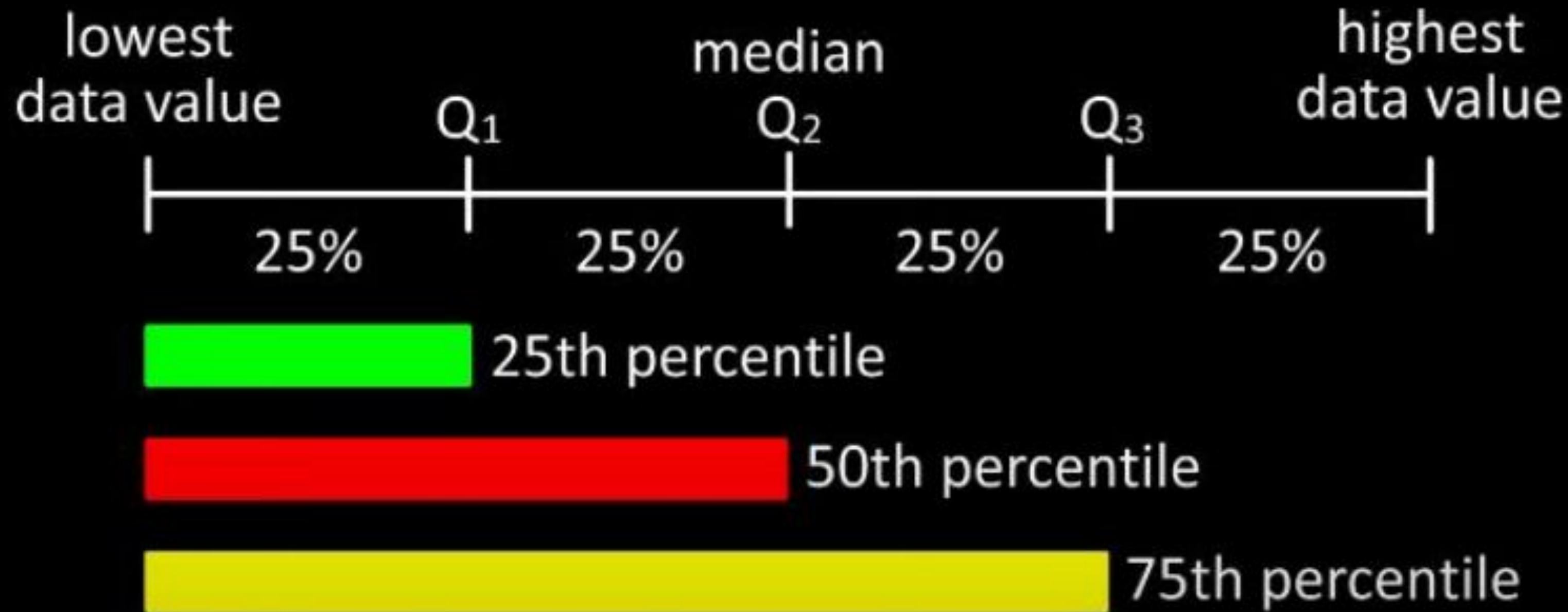
- `isnull().sum()` → Counts missing values in each column
- `dropna()` → Removes rows that contain missing values
- `fillna(df.mean())` → Replaces missing values with the average of each column

Box Plot

Quartiles, The Interquartile Range, And Outliers

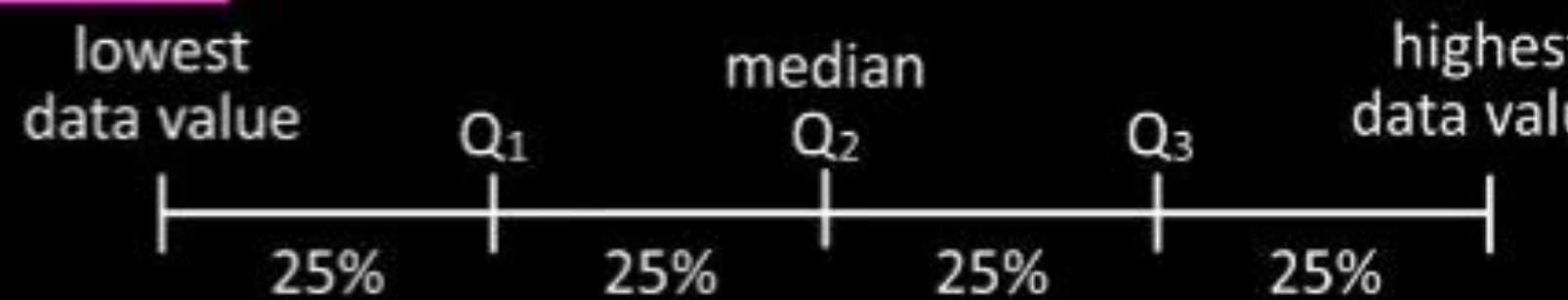
quartiles

- divide a data set into 4 equal groups
- are marked Q_1 , Q_2 , and Q_3



Quartiles, The Interquartile Range, And Outliers

Identifying data values that correspond to Q_1 , Q_2 , Q_3



data set

14 18 15 6 21 9 16 12 23 7 11 19 24 5 31 10 27 17 25 8

arrange the data smallest to largest

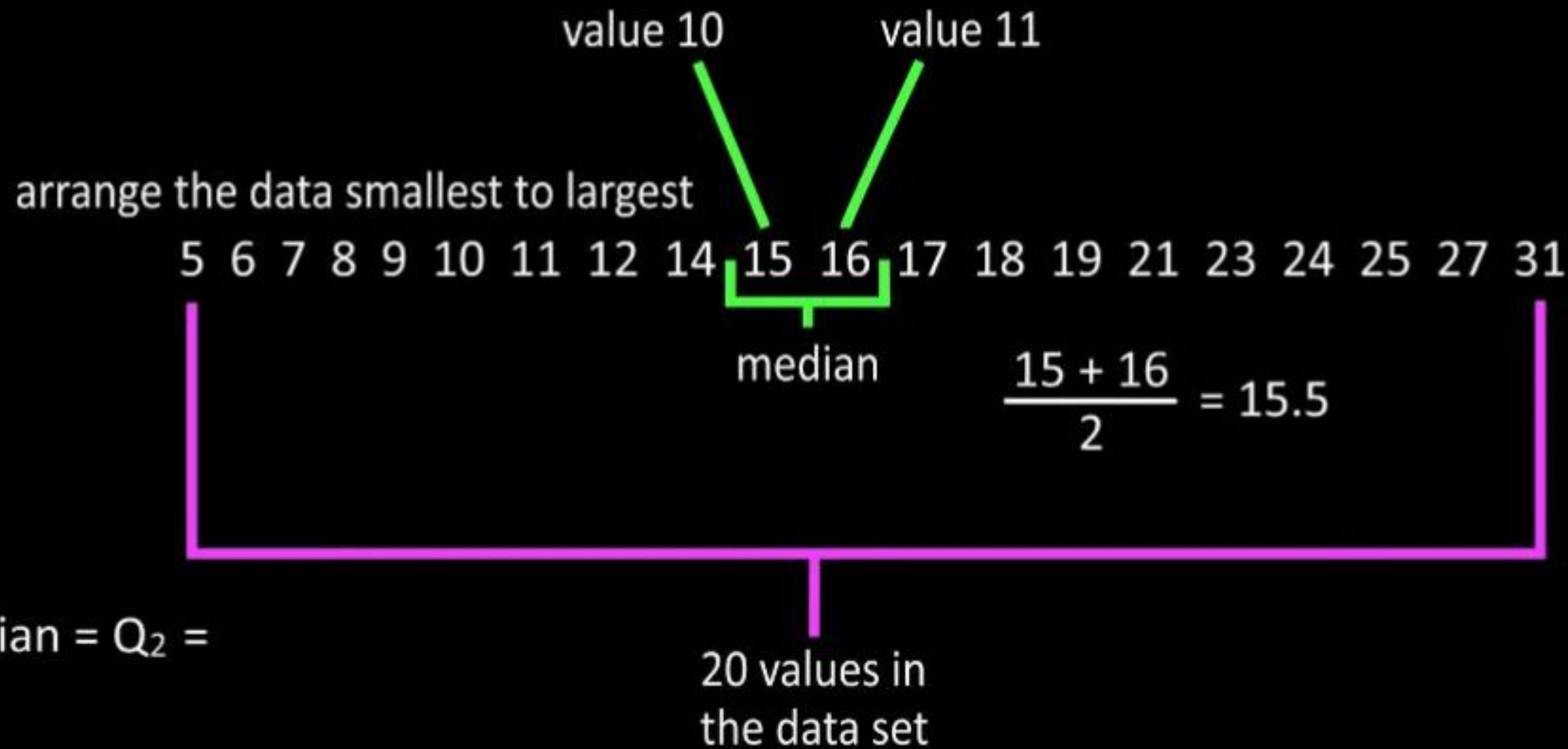
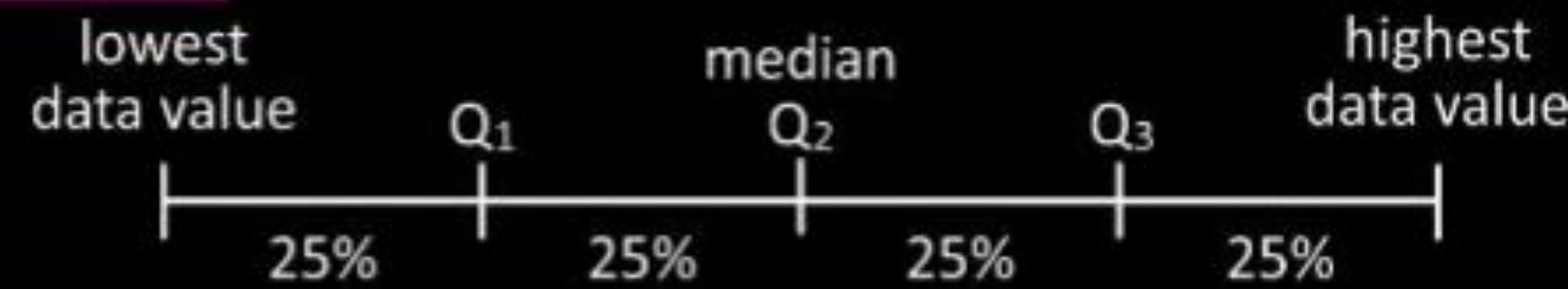
5 6 7 8 9 10 11 12 14 15 16 17 18 19 21 23 24 25 27 31

median = Q_2 =

20 values in
the data set

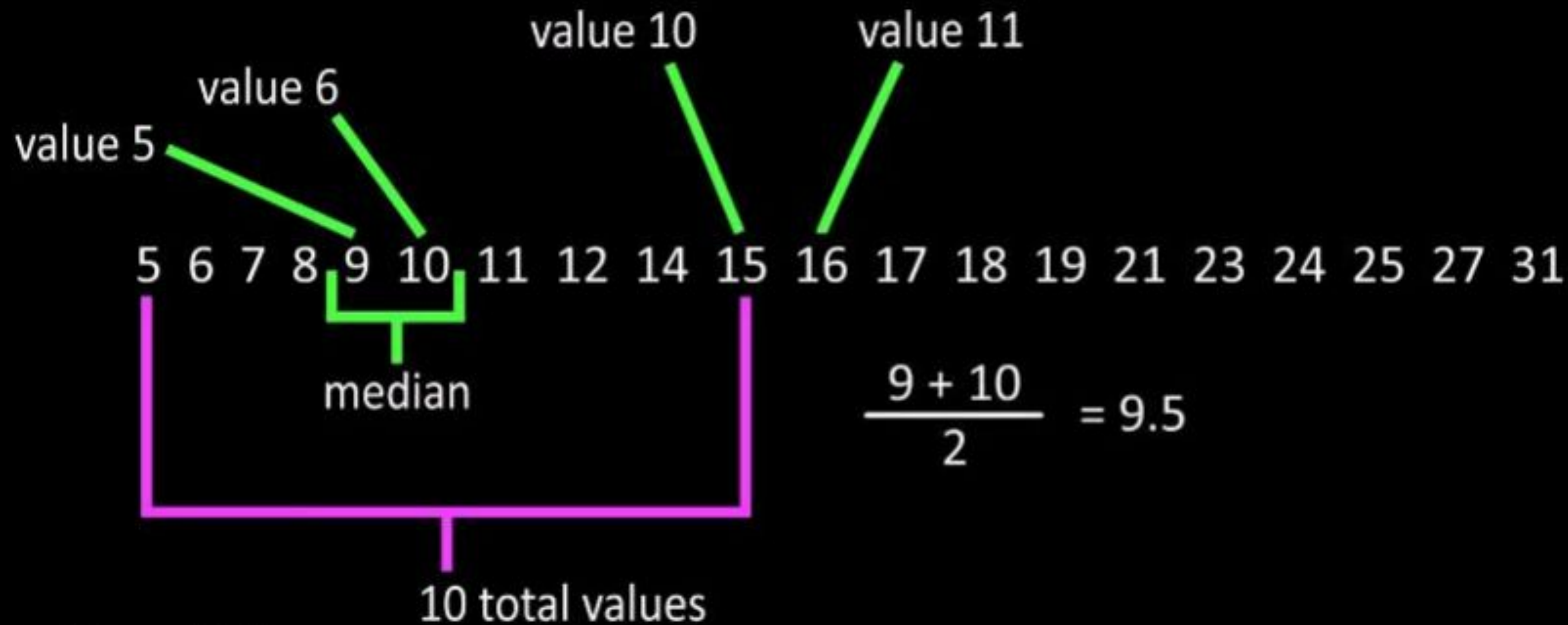
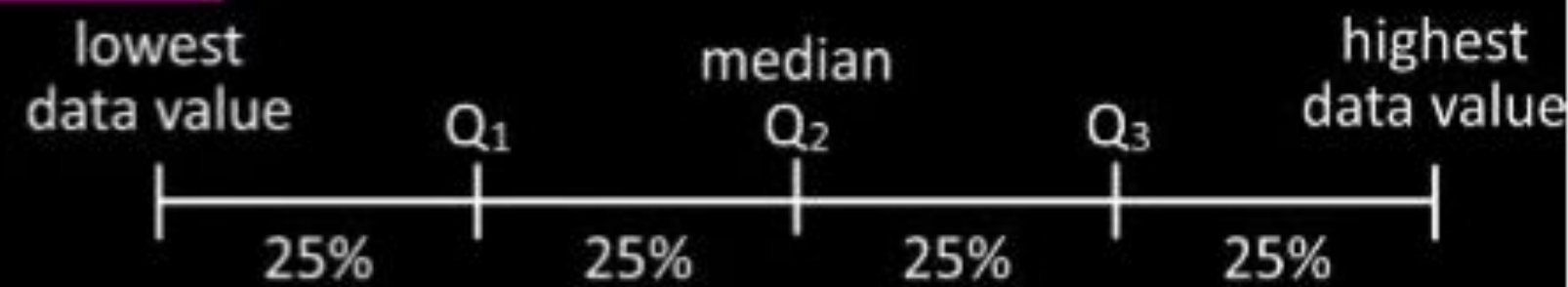
Quartiles, The Interquartile Range, And Outliers

Finding data values that correspond to Q_1 , Q_2 , Q_3



Quartiles, The Interquartile Range, And Outliers

Finding data values that correspond to Q_1 , Q_2 , Q_3



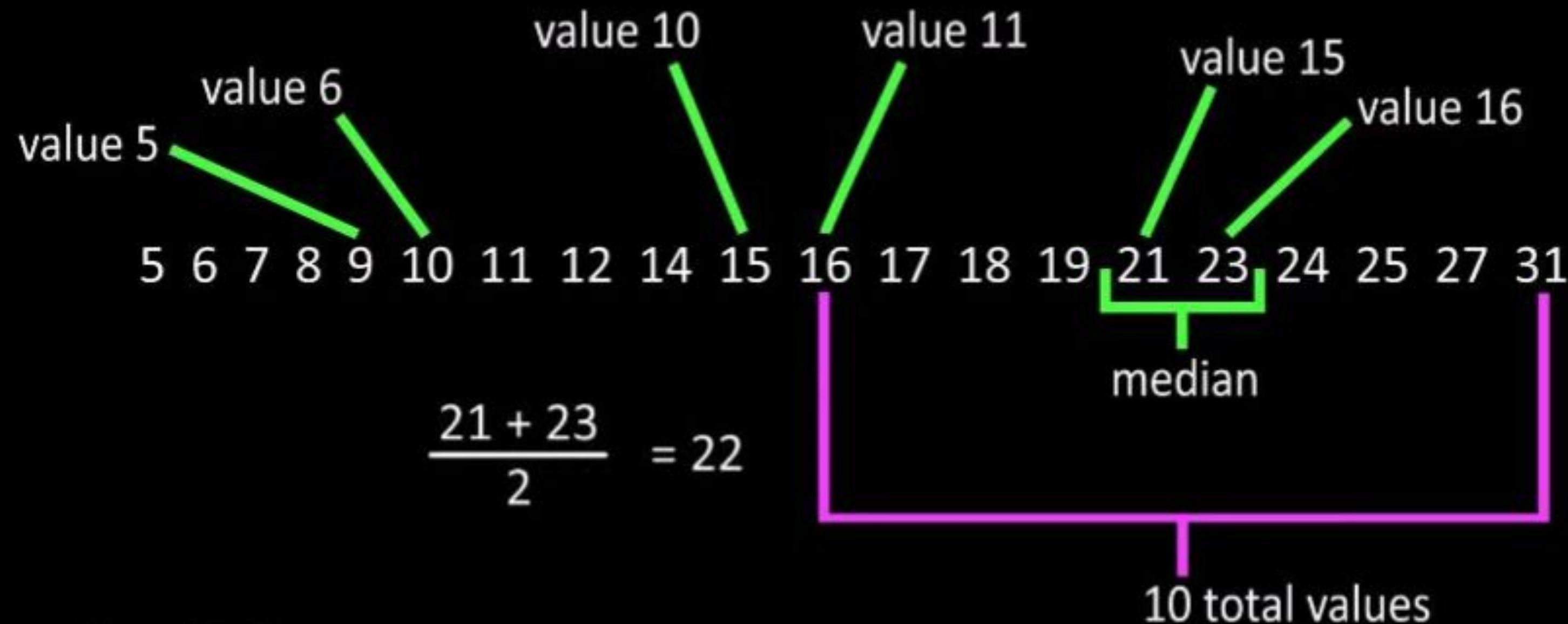
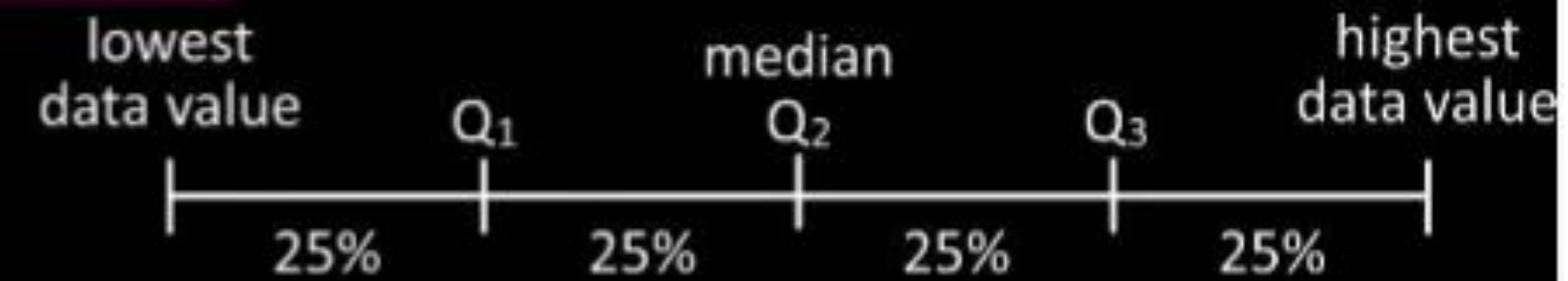
$$\frac{9 + 10}{2} = 9.5$$

median = Q_2 = 15.5

median of values that fall below Q_2 = Q_1 = 9.5

Quartiles, The Interquartile Range, And Outliers

Finding data values that correspond to Q_1 , Q_2 , Q_3



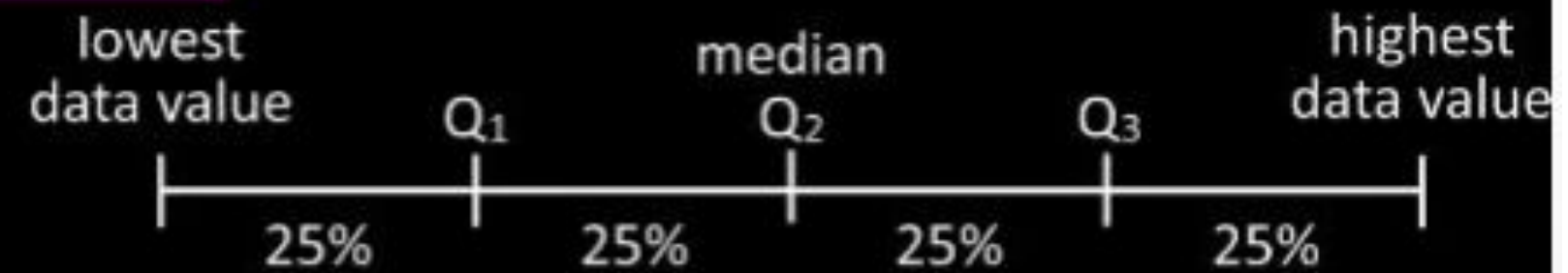
median = Q_2 = 15.5

median of values that fall below Q_2 = Q_1 = 9.5

median of values that fall above Q_2 = Q_3 =

Quartiles, The Interquartile Range, And Outliers

Finding data values that correspond to Q_1 , Q_2 , Q_3



5 6 7 8 9 10 11 12 14 15 16 17 18 19 21 23 24 25 27 31

$$Q_1 = 9.5$$

$$Q_2 = 15.5$$

$$Q_3 = 22$$

Interquartile Range (IQR)

- difference between 3rd and 1st quartiles

$$IQR = Q_3 - Q_1$$

$$IQR = 22 - 9.5 = 12.5$$

Quartiles, The Interquartile Range, And Outliers

Finding data values that correspond to Q_1 , Q_2 , Q_3

Interquartile Range (IQR)

$$IQR = Q_3 - Q_1$$

$$IQR = 22 - 9.5 = 12.5$$

$$Q_1 = 9.5$$

$$Q_2 = 15.5$$

$$Q_3 = 22$$

5 6 7 8 9 10 11 12 14 15 16 17 18 19 21 23 24 25 27 31

5 6 7 8 9 10 11 12 14 15 16 18 19 21 23 24 25 27 31 170

IQR to identify outliers

$$Q_1 = 9.5$$

$$Q_3 = 23.5$$

$$IQR = 23.5 - 9.5 = 14$$

range of (-11.5 to 44.5)

$$\text{multiply IQR by } 1.5 = 14 \times 1.5 = 21$$

- any number outside of the range is an outlier

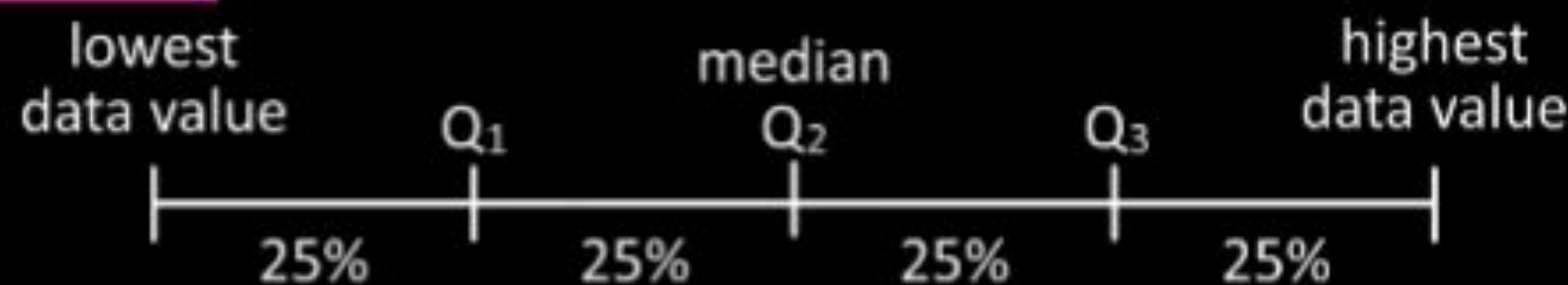
$$Q_1 - 21 = -11.5$$

outlier =

$$Q_3 + 21 = 44.5$$

any data value smaller than $Q_1 - 1.5(IQR)$

any data value larger than $Q_3 + 1.5(IQR)$



Working with Excel Files in Python

- Python provides several libraries to work with **Excel files**.
- Commonly used libraries include:
 - **Pandas** - for data analysis and manipulation
 - **Openpyxl** - for reading and writing Excel **.xlsx** files
- Installing the Required Library
 - **pip install openpyxl**

Reading an Excel File using Pandas

```
import pandas as pd  
df1 = pd.DataFrame()  
print(df1)
```

Writing Data to an Excel File

```
df.to_excel("output.xlsx", index=False)
```

Openpyxl Example

```
xls-read-write-1.py - C:\Users\zasha\OneDrive\Desktop\SE412-Fall2020\Python-CSV\xls-read-write-1.py (3.9.0)
File Edit Format Run Options Window Help
from openpyxl import Workbook
wb = Workbook()

# grab the active worksheet
ws = wb.active

# Data can be assigned directly to cells
ws['A1'] = 42

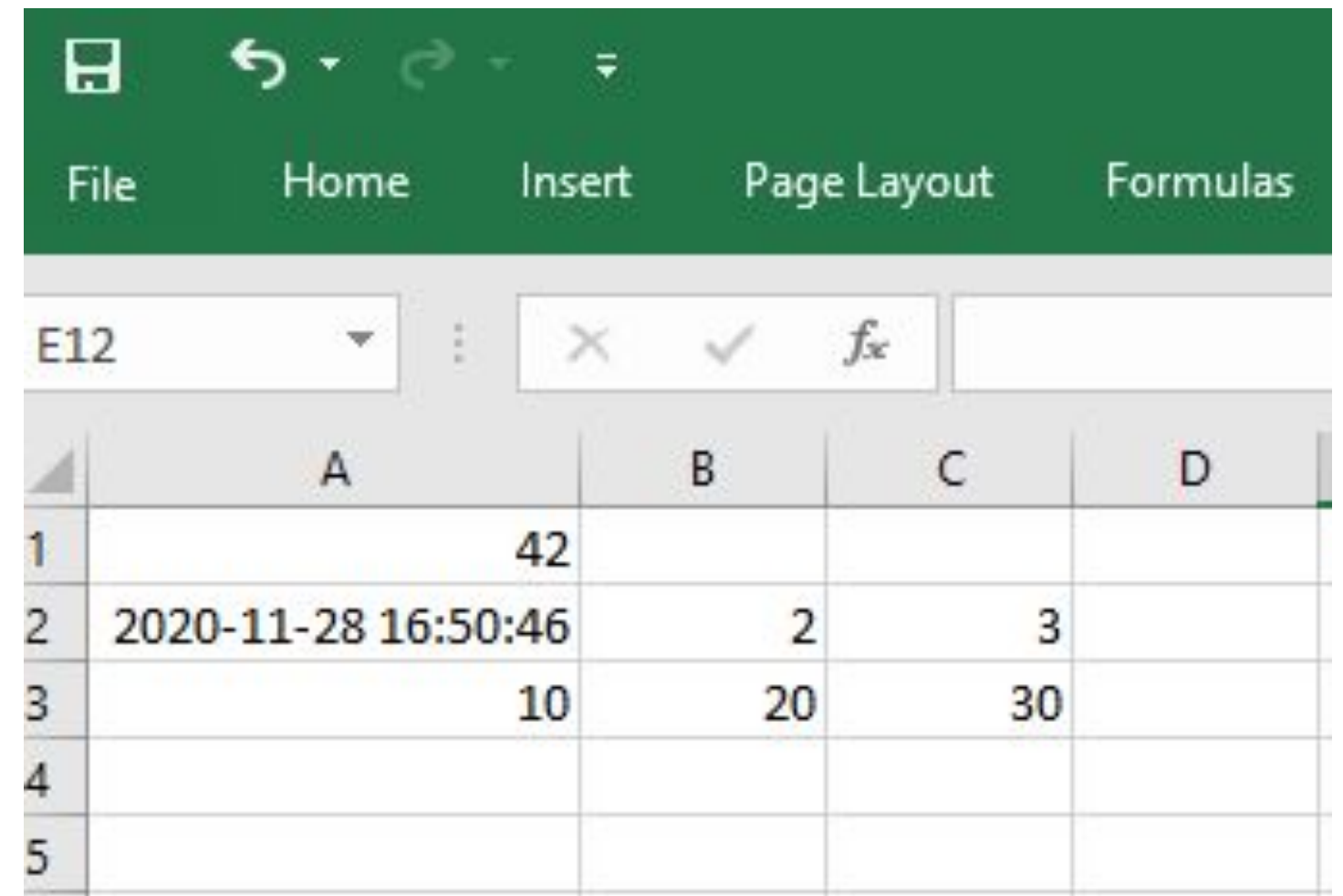
# Rows can also be appended
ws.append([1, 2, 3])

# Python types will automatically be converted
import datetime
ws['A2'] = datetime.datetime.now()

ws.append([10, 20, 30])

# Save the file
wb.save("sample.xlsx")
```

Ln: 20 Col: 0



	A	B	C	D
1	42			
2	2020-11-28 16:50:46	2	3	
3	10	20	30	
4				
5				

References

- CSV File Reading and Writing

<https://docs.python.org/3/library/csv.html>

- Reading and Writing CSV Files in Python

<https://realpython.com/python-csv/>

- pandas.read_csv

https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html

- Working with Excel Files in Python

<https://www.python-excel.org/>

- openpyxl - A Python library to read/write Excel 2010 xlsx/xlsm files

<https://openpyxl.readthedocs.io/en/stable/>